

TALLER PRÁCTICO · ESPECIALIZACIÓN EN CIENCIA DE DATOS · UNI

MLOps en la práctica

Del notebook a producción

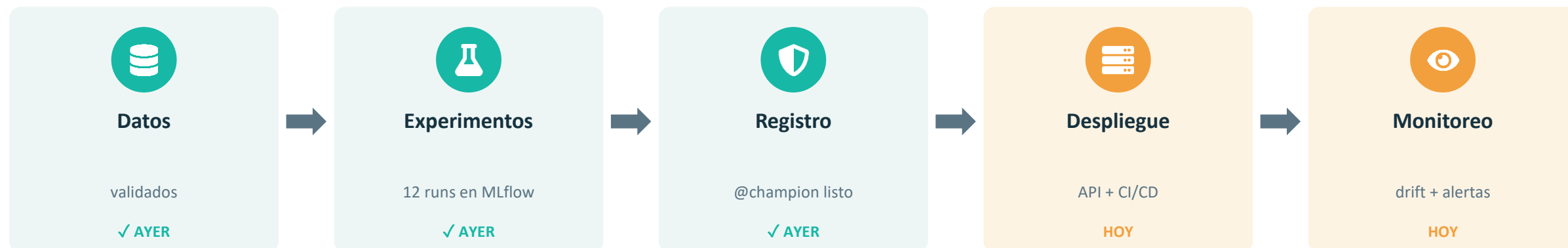
DÍA 2 · Del modelo al servicio monitoreado

APIs con FastAPI · Docker · CI/CD · Monitoreo y drift

Hoy tu modelo atiende clientes — y aprendemos a vigilarlo
Jordan King Rodriguez Mallqui — Doctorando en Ciencias e Ingeniería Estadística



Ayer construimos la fábrica; hoy abrimos la tienda



LA PREGUNTA DE HOY

El call center no importa pipelines de Python. **¿Cómo convertimos “un modelo con alias” en un servicio que cualquier sistema consulta, que se despliega igual en cualquier servidor, que se prueba solo en cada cambio... y que grita cuando el mundo cambia?**

¿Cómo consume el negocio las predicciones?



POR LOTES (BATCH)

Un job corre cada noche, puntúa a los 2.4 M de clientes y deja la lista en una tabla.

- Ideal: campañas, reportes, scoring masivo
- Simple de operar, barato, tolerante a fallas
- Latencia: horas — el dato puede envejecer



EN LÍNEA (API REST)

Un servicio responde en milisegundos cuando el CRM pregunta por UN cliente.

- Ideal: decisiones en el momento (el cliente está EN la llamada)
- Requiere: disponibilidad, latencia, contrato de datos
- **Es lo que construiremos hoy (y sirve para ambos)**

Regla práctica: empieza batch si el negocio lo tolera; pasa a online cuando la decisión no pueda esperar. AndesTel necesita ambos.

L3

LAB 3 · 65 MIN

Serving con FastAPI (parte B)

OBJETIVO exponer el @champion como API REST con contrato de datos validado — como lo consumiría el CRM.

EN ESTE LAB

- Anatomía de app.py: lifespan, carga única del modelo por alias
- El contrato Pydantic: tipos, rangos y categorías permitidas
- Levantar el servicio y consultarlo con requests
- Romperlo a propósito: leer errores 422 como un integrador
- /docs: la documentación interactiva que no escribiste
- Reto: /predict_lote ordenado por riesgo (a quién llamar primero)



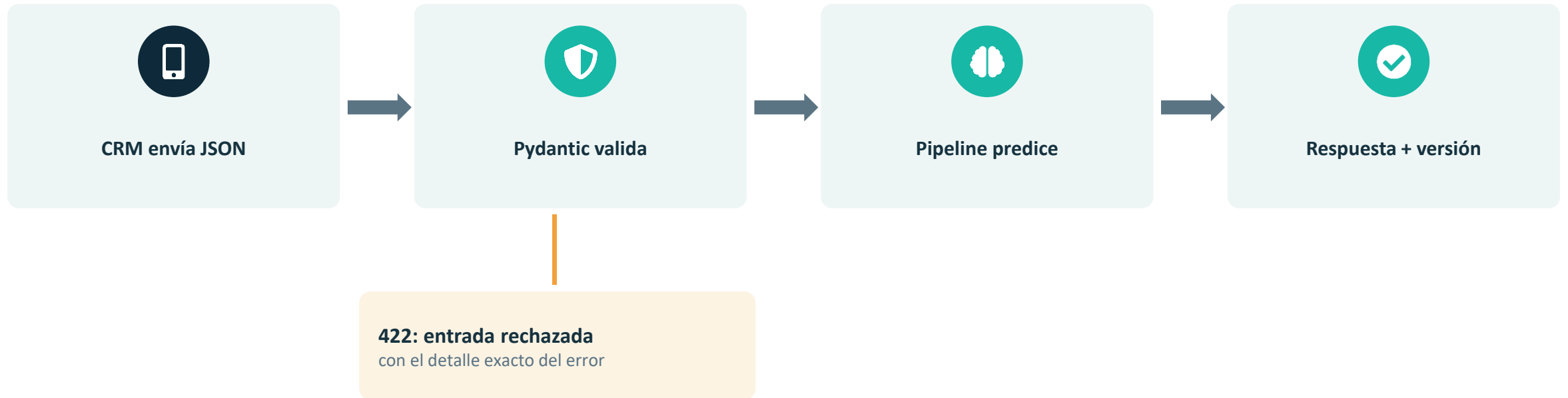
CUADERNO
nb3_registry_
serving.ipynb (Parte B)



¿VAS RÁPIDO?

Latencia p50/p95 con 200 requests —
¿cumples un SLA de 100 ms? (Reto 4)

El contrato de datos es tu primera línea de defensa



- ✓ **Los datos malos NO llegan al modelo** — edad “cuarenta”, plan inexistente, campo faltante: rechazados en la puerta con explicación.
- ✓ **El modelo se carga UNA vez, al arrancar** — cargarlo por request es el error de latencia más común en APIs de ML.
- ✓ **Cada respuesta declara su versión de modelo** — auditable: “¿qué modelo decidió llamar a este cliente?” siempre tiene respuesta.

Docker: “en mi máquina funciona” — y en todas

EL PROBLEMA

Tu API corre en tu laptop con Python 3.11, sklearn 1.8, tus variables de entorno y tu suerte.

El servidor de producción tiene Python 3.9, otro sklearn y ninguna suerte. El despliegue muere un viernes a las 7 pm.

El modelo cambia el problema: no solo despliegas código — despliegas código + modelo + versiones exactas de librerías que deben calzar con las del entrenamiento.

LA SOLUCIÓN: LA IMAGEN

python:3.11-slim
base idéntica en todas partes

requirements.txt
librerías pinneadas, instaladas una vez

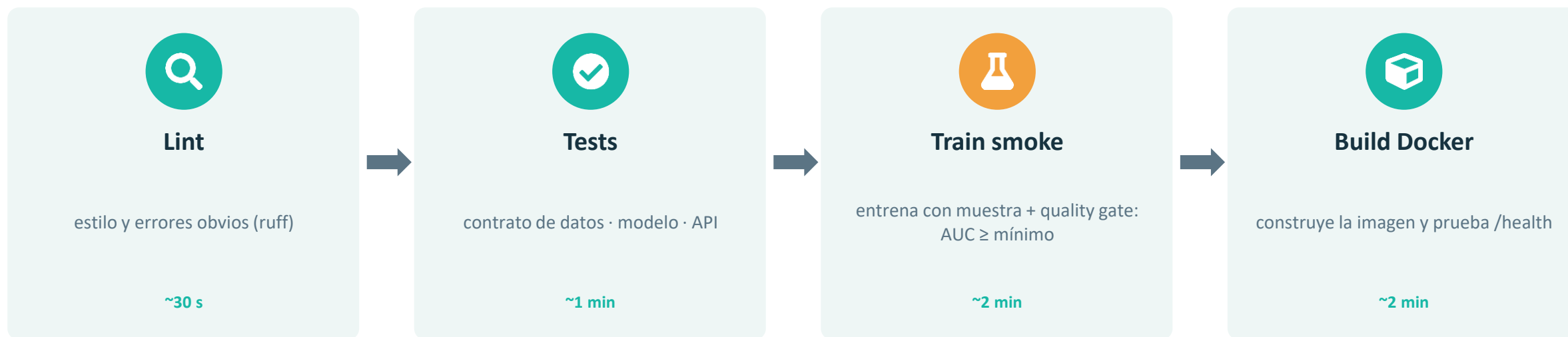
src/ + params.yaml
tu código y configuración

models/modelo.joblib
el modelo entrenado, DENTRO de la imagen

docker run → el mismo servicio, bit a bit, en tu laptop, el servidor de la UNI o la nube

CI/CD: cada cambio se prueba solo, antes de dañar producción

En cada push a GitHub, un robot ejecuta el pipeline. Si algo falla, el cambio no avanza. Para ML, el pipeline prueba más que código:



El “quality gate” es la idea nueva para ML: el CI re-entrena un modelo pequeño y FALLA si no supera el AUC mínimo acordado (vive en params.yaml). Un cambio que rompe el pipeline de entrenamiento o degrada el modelo se detecta en el pull request — no seis meses después, en producción.

L4

LAB 4 · 60 MIN

Tu repo con Docker + CI/CD en GitHub Actions

OBJETIVO publicar tu propio repo desde la plantilla del taller y ver a GitHub Actions probar, entrenar y dockerizar tu proyecto.

EN ESTE LAB

- Recorrido por la plantilla: src/, tests/, params.yaml, Dockerfile
- Sube la plantilla a TU GitHub (git init → push)
- Mira el pipeline correr en la pestaña Actions: 4 etapas en verde
- Sabotaje controlado: rompe una columna, push, y CI lo atrapa
- Arréglalo y recupera el verde
- Este repo ES tu punto de partida para proyectos reales



CUADERNO

plantilla-proyecto-
mlops/ (README = guía)



¿VAS RÁPIDO?

Baja el quality gate a 0.90 y mira fallar el gate.
Luego: publica metrics.json como artifact

Tu API está verde. Tu modelo está mintiendo.

Es marzo de 2026. Desde el despliegue: tarifas +18%, campaña agresiva de captación, más caídas de red. El modelo fue entrenado en un mundo que ya no existe.



Lo que ve infraestructura

API 200 OK · 40 ms · 0 errores
Todos los dashboards en verde



Lo que nadie está viendo

Los datos de entrada ya no se parecen a los de entrenamiento: las predicciones se degradan en silencio

ground truth delay: sabrás el churn real recién en 3 meses. Para entonces la pérdida ya ocurrió. Necesitas señales que NO esperen la verdad: las distribuciones de los datos y de las predicciones.

L5

LAB 5 · 45 MIN

Monitoreo y data drift con Evidently

OBJETIVO detectar que el mundo cambió ANTES de que llegue la verdad del terreno — y traducirlo a soles.

EN ESTE LAB

- Primera señal: la distribución de las predicciones se movió
- Reporte de drift con Evidently: referencia 2025 vs producción 2026
- Leer el reporte como detective: ¿qué cambió? ¿coincide con el negocio?
- Medir la degradación real (AUC, recall) y convertirla a S/
- chequeo_drift(): la función lista para un job programado
- Política de alerta y re-entrenamiento: umbrales → acciones



CUADERNO

nb4_monitoreo_
drift.ipynb



¿VAS RÁPIDO?

Monitoreo semanal simulado: 4 lotes, ¿en qué semana disparas la alerta? (Reto 3)

Monitoreo = política explícita, no dashboard decorativo

SEÑAL

% columnas con drift > 30%

Drift en features top de importancia

Media de predicciones se mueve > 5 pts

AUC real cae > 0.03 (cuando llegue)

Recall en producción < 0.50

ACCIÓN AUTOMÁTICA O HUMANA

Revisar con negocio qué cambió

Investigación inmediata

Alerta al equipo de DS

Re-entrenar con datos recientes

Re-entrenar + revisar umbral



Re-entrenar cierra el ciclo: mismo pipeline del Lab 1, comparación en MLflow contra el champion, y si gana... mover el alias. Todo lo del taller, junto.

Proyecto integrador: MLOps en TU trabajo

Para consolidar (individual o en pareja, ~4 h de tu semana). Elige un problema de TU empresa o investigación y móntalo sobre la plantilla:

1

Adapta la plantilla

tu dataset, tu contrato de datos,
tus features en params.yaml

2

Entrena con tracking

≥ 6 configuraciones en MLflow +
criterio de selección escrito

3

Registra y sirve

@champion + esquema Pydantic
de tu caso

4

CI en verde

push a GitHub con las 4 etapas
pasando

5

Plan de monitoreo

tu tabla señal → acción +
chequeo_drift adaptado



Entregable sugerido: el enlace a tu repo con el CI en verde + un README de 1 página que explique tus decisiones (features, criterio de selección, umbral, política de monitoreo). Con eso tienes un portafolio MLOps real — y una propuesta lista para tu jefe el lunes.

CIERRE DEL TALLER

En 8 horas: el ciclo completo, con tus manos

- ✓ Un pipeline reproducible que tolera el mundo real (nulos, categorías nuevas, datos sucios)
- ✓ 12+ experimentos rastreados y un campeón elegido con criterio auditable
- ✓ Un modelo versionado, promovido por alias y servido por API con contrato validado
- ✓ Un repo con CI/CD que entrena, prueba y dockeriza en cada push
- ✓ Un sistema de monitoreo que detecta el drift antes que la verdad del terreno

PARA SEGUIR

Proyecto integrador (guía en el material) · checklist de madurez MLOps · docs de MLflow, FastAPI y Evidently enlazadas en el repo

LA IDEA FINAL

MLOps no es comprar herramientas: es una disciplina de equipo. Empieza el lunes con UN paso: tracking de experimentos.